

SQL, data flow i rola Data Engineera

2026-07-12

Lesson 01

■ SQL, data flow i rola Data Engineera

■ Pierwsza lekcja merytoryczna

Cel: nie tylko napisac query, ale wiedziec co ono znaczy.

■ Zanim zaczniemy SQL

W tej lekcji pilnujemy jednego pytania:

Co oznacza wynik query?

```
tabela = dane w wierszach i kolumnach  
JOIN   = połączenie tabel  
grain  = co oznacza jeden rekord
```

Warstwy platformy danych wroca później. Teraz skupiamy się na małym modelu danych.

■ Problem biznesowy

Sklep pyta:

```
Ile zarobilismy?  
Ktore produkty sprzedaja sie najlepiej?  
Ktorzy klienci wracaja?  
Czy metryka jest policzona tak samo jak tydzien temu?
```

SQL odpowiada na pytanie.

Data Engineering sprawia, ze odpowiedz jest powtarzalna i zaufana.

Application

- > writes orders
- > writes order_items
- > zapisuje dane transakcyjne

Production DB

- > stores current transactional state

Data platform

- > snapshots / CDC
- > cleans data
- > models metrics

■ Gdzie w tym SQL?

SQL pojawia sie w wielu miejscach:

- sprawdzenie danych zrodlowych,
- laczenie tabel,
- liczenie metryk,
- testy jakosci,
- odpowiedz interview.

SQL to nie tylko raport. To jezyk kontroli danych.

■ E-commerce dataset

Bedziemy uzywac jednego case'u:

```
customers  
orders  
order_items  
products
```

Jeden dataset pozwala widziec progres: tabele -> joiny -> metryki -> walidacja.

■ Najpierw relacje

Zanim zrobisz JOIN – musisz wiedzieć PO CZYM łączyc tabele.

```
PK = Primary Key: unikalny identyfikator wiersza w tej tabeli  
FK = Foreign Key: odwołanie do PK w innej tabeli
```

JOIN łączy tabele po PK ↔ FK.

■ Relacje w datasetcie

```
customers  customer_id (PK)
           ↳ orders    customer_id (FK) order_id (PK)
                               ↳ order_items order_id (FK)

products   product_id (PK)
           ↳ order_items product_id (FK)
```

Jesli polaczysz po zlej kolumnie:

```
query odpali sie poprawnie,
ale wynik bedzie biznesowo bezsensowny.
```

SQL robi dwie różne rzeczy:

```
-- DDL: zmienia STRUKTURE tabeli
CREATE TABLE test_orders (...);
ALTER TABLE test_orders ADD COLUMN source_file TEXT;
DROP TABLE IF EXISTS test_orders;

-- DML: zmienia DANE w tabeli
SELECT * FROM orders;
INSERT INTO test_orders VALUES (...);
UPDATE test_orders SET status = 'paid';
DELETE FROM test_orders WHERE id = 1;
```

■ DDL vs DML: zapamiętaj

W tej lekcji:

```
DDL = zmieniasz strukture tabeli testowej  
DML = czytasz albo zmieniasz dane
```

Uwaga: ten sam INSERT dwa razy = duplikaty. Na razie tylko zapamiętaj: po INSERT/UPDATE zawsze sprawdź wynik SELECT-em.

■ Grain: najważniejsze słowo

Grain = co oznacza jeden rekord?

Przykład:

Tabela	Grain = jeden rekord oznacza...
-----	-----
customers	jednego klienta
orders	jedno zamówienie
order_items	jedna pozycja zamówienia
products	jeden produkt

■ Dlaczego grain ratuje wynik?

Rozny grain = rozna liczba wierszy:

orders			order_items			
order_id	status		order_item_id	order_id	qty	price
1	paid	→	1	1	2	500
		└→	2	1	1	50
2	paid	→	3	2	3	20

COUNT(orders) = 2
COUNT(order_items) = 3 ← zawsze sprawdz przed i po joinie!

Join `orders → order_items` zwiększa liczbę wierszy.

■ Grain: dobra praktyka

Dobra praktyka:

- przed joinem: `SELECT COUNT(*) FROM orders`
- po joinie: `SELECT COUNT(*) FROM orders JOIN order_items USING (order_id)`
- napisz grain wyniku jednym zdaniem ZANIM zaczniesz pisać query

Nie rob:

- nie zakładaj, że join nie zmienił grain
- nie licz `SUM(unit_price)` bez sprawdzenia czy rekord się nie podwoił

■ Grain i JOIN wizualnie

orders			order_items			
order_id	status		item_id	order_id	qty	price
101	paid	----->	1001	101	1	6200
101	paid	----->	1002	101	2	1200
102	paid	----->	1003	102	1	900

Po JOIN orders -> order_items:
wynik = 3 wiersze (grain: 1 pozycja zamowienia)

TAK – JOIN zmienił grain wyniku:

przed JOIN:	grain = 1 zamówienie	(orders: 2 wiersze)
po JOIN:	grain = 1 pozycja	(order_items: 3 wiersze)

Order 101 pojawia się dwa razy, bo ma 2 pozycje. Jeśli teraz policzysz `COUNT(*)` – liczysz pozycje, nie zamówienia.

■ Query może być poprawne składniowo i złe biznesowo

SQL może się odpalić i dalej dać złą liczbę.

Błąd techniczny?

Query się nie odpala.

Błąd biznesowy?
innego.

Query się odpala, ale liczba znaczy coś

Prosty test:

Czy umiesz powiedzieć jednym zdaniem,
co oznacza 1 wiersz wyniku?

Problem	Dlaczego wynik jest zły?
-----	-----
orders JOIN order_items sie powtarza canceled w sales SUM(unit_price) pozycji COUNT(*) po JOIN brak grain wyniku customer/order/item	1 order ma wiele pozycji, wiec order cancelled oznacza: klient nie kupil cena jednej sztuki != wartosc calej po JOIN liczysz pozycje, nie zamowienia nie wiesz czy 1 wiersz =

Przed query zapytaj:

1. Jakie pytanie biznesowe odpowiadam?
2. Co oznacza jeden rekord wejscia?
3. Co oznacza jeden rekord wyniku?
4. Jak definiuje metryke?
5. Jak sprawdzę, że wynik ma sens?

■ SQL ma dwa porządki

Piszesz zwykle tak:

```
SELECT ...  
FROM ...  
WHERE ...  
GROUP BY ...  
HAVING ...  
ORDER BY ...  
LIMIT ...
```

Ale logicznie baza myśli inaczej.

Silnik bazodanowy przetwarza query w tej kolejności:

```
FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT
```

FROM	skąd bierzesz dane
WHERE	które rekordy wyrzucasz
GROUP BY	po czym grupujesz
HAVING	filtr gotowych grup
SELECT	co pokazujesz
ORDER BY	kolejność wyniku
LIMIT	ile wierszy

Alias z **SELECT** nie działa w **WHERE**. **SELECT** jest przetwarzany dopiero po **WHERE**.

Zdanie do zapamiętania:

```
FROM WHERE GROUPS HAVE SELECTED ORDERS LIMITS
```

Po polsku:

"Skąd grupy mają wybrane zamówienia z limitami."

■ Build 1: skad dane i ktore rekordy

```
FROM / JOIN  ← skad bierzesz dane  
-> WHERE    ← ktore rekordy wyrzucasz
```

Najpierw wybierasz z jakich tabel i ktore rekordy chcesz. Dopiero potem grupujesz i liczysz metryki.

FROM	WHERE	GROUP BY	HAVING	SELECT	ORDER BY
LIMIT					
skad	ktore	po czym	majac	co	posortuj ile

FROM WHERE GROUPS HAVE SELECTED ORDERS LIMITS

Po polsku: "Skad grupy maja wybrane zamowienia z limitami."

■ Build 2: grupy i filtr na grupach

```
FROM / JOIN
-> WHERE    ← filtr surowych rekordow
-> GROUP BY
-> HAVING   ← filtr gotowych grup
```

WHERE filtruje pojedyncze rekordy przed grupowaniem. **HAVING** filtruje gotowe grupy po agregacji.

```
FROM WHERE GROUPS HAVE SELECTED ORDERS LIMITS
Po polsku: "Skad grupy maja wybrane zamowienia z limitami."
```

■ Build 3: dopiero teraz SELECT

```
FROM / JOIN  
-> WHERE  
-> GROUP BY  
-> HAVING  
-> SELECT ← dopiero tutaj powstają aliasy i nazwy kolumn  
-> ORDER BY  
-> LIMIT
```

Alias z **SELECT** nie istnieje jeszcze w **WHERE**. **WHERE** jest przetwarzany wcześniej.

■ Najczęstszy błąd: alias w WHERE

```
SELECT
    quantity * unit_price AS gross_line_value
FROM order_items
WHERE gross_line_value > 100;
```

Co jest nie tak?

■ Poprawka: powtorz wyrażenie w WHERE

```
SELECT
    quantity * unit_price AS gross_line_value
FROM order_items
WHERE quantity * unit_price > 100;
```

gross line value nie istnieje jeszcze w momencie WHERE. Baza przetwarza WHERE zanim dotrze do SELECT. Dlatego musisz powtorzyć wyrażenie.

```
WHERE jest przed SELECT.
Alias powstaje dopiero w SELECT.
```

■ Najczęstszy błąd: WHERE zamiast HAVING

```
SELECT customer_id, COUNT(*) AS orders_count
FROM orders
WHERE COUNT(*) > 1
GROUP BY customer_id;
```

Co jest nie tak?

■ Poprawka: HAVING filtruje grupy

```
SELECT customer_id, COUNT(*) AS orders_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

COUNT(*) to wynik agregacji. Powstaje dopiero po GROUP BY. WHERE nie może filtrować czegoś, co jeszcze nie istnieje.

```
HAVING jest po GROUP BY.
WHERE jest przed GROUP BY.
```

■ Mini brief przed query

Zanim napiszesz SQL, zapisz 5 rzeczy:

1. Brief:
Ile revenue ma kazdy customer?
2. Tabele:
customers, orders, order_items
3. Grain wyniku:
1 wiersz = 1 klient
4. Metryka:
revenue = SUM(quantity * unit_price)
5. Check:
1 customer_id wystepuje max 1 raz w wyniku

■ Mini brief: po co?

To jest prosty kontrakt:

- co liczysz,
- z czego,
- na jakim grain,
- jak sprawdzisz wynik.

■ Revenue: definicja zanim SQL

Zła definicja:

```
Revenue = amount
```

Lepsza definicja:

```
Revenue = sum(order_items.quantity * order_items.unit_price)
Tylko zamówienia ze statusem paid.
Nie licz cancelled, jeśli nie ustalono inaczej.
Grain wyniku: jeden wiersz na klienta.
```

ZLA definicja:

revenue = amount

→ co to jest "amount"? per order? per item? brutto czy netto?

DOBRA definicja:

revenue = SUM(quantity * unit_price)

filtry: status = 'paid', bez cancelled i refunded

grain: 1 wiersz na klienta

CHECKI JAKOSCI:

customer_id NOT NULL

revenue >= 0

COUNT = COUNT(DISTINCT customer_id)

■ Revenue: co sprawdzaja checki?

Te checki mowia:

- wynik ma wlasciciela,
- revenue nie jest ujemne,
- jest 1 wiersz na klienta.

CHECKI JAKOSCI:

check

dłaczego?

tylko zamówienia paid
sprzedazy

nie licz cancelled jako

COUNT przed i po JOIN

wiesz czy JOIN zmienił grain

Metryka bez definicji, filtra statusu, grain i checkow jest tylko liczba.

■ CTE: bonus

CTE = **WITH** – nazwany krok wewnątrz query.

Bez CTE – wszystko w jednym bloku, trudno czytać:

```
SELECT customer_id, SUM(quantity * unit_price) AS revenue
FROM orders
JOIN order_items USING (order_id)
WHERE status = 'paid'
GROUP BY customer_id
HAVING SUM(quantity * unit_price) > 0;
```

Im dłuższe query, tym trudniej zobaczyć co robi każdy krok.

```
WITH paid_orders AS (  
    SELECT order_id, customer_id  
    FROM orders  
    WHERE status = 'paid'  
),  
line_revenue AS (  
    SELECT order_id, quantity * unit_price AS line_value  
    FROM order_items  
)  
SELECT customer_id, SUM(line_value) AS revenue  
FROM paid_orders  
JOIN line_revenue USING (order_id)  
GROUP BY customer_id;
```

Wynik identyczny. Każdy krok ma nazwę – łatwiej czytać i debugować. CTE nie tworzy tabeli, istnieje tylko w tym query.

Performance: CTE na tym przykładzie nic nie zmienia. Optimizer (SQLite, DuckDB, Postgres) wkleja CTE inline i wykonuje ten sam plan. CTE to narzędzie czytelności, nie optymalizacji.

■ Walidacja wyniku

Minimalne checki:

```
customer_id nie jest NULL  
revenue >= 0  
jeden wiersz na klienta  
tylko zamówienia paid  
wyjasniona liczba rekordow przed i po joinie
```

Bez walidacji mamy tylko nadzieje, nie pewny wynik.

■ Java → DE: analogia dla backendowca

Java/backend

kontrakt endpointu
DTO/encja
request id
logika serwisu
unit test
konsument API

Data Engineering

kontrakt danych
schemat/tabela
order id
logika transformacji
check walidacyjny SQL
odbiorca danych

Zadania do pracy domowej

1. Obejrzyj dane i policz rekordy w 4 tabelach.
2. Policz query biznesowe: paid, statusy, revenue.
3. Zbuduj finalne query `customer order summary`.
4. Dopisz 4 proste validation checks.
5. Opisz grain tabel i finalnego wyniku.
6. Oddaj pliki `01_solution.sql` - `05_interview_answers.md`.

Najpierw uruchom bazę:

Na Windowsie: WSL Ubuntu, nie PowerShell.

```
cd student/lab  
sqlite3 lesson_01.db < schema.sql  
sqlite3 lesson_01.db < seed_data.sql
```

```
schema.sql      = tworzy tabele  
seed_data.sql   = ładuje gotowe dane
```

■ Praca domowa: pomocnicze laby

Potem użyj ćwiczeń pomocniczych, jeśli potrzebujesz prowadzenia krok po kroku:

1. lab/04_sql_basics_extra_tasks.sql -> zobaczenie danych
2. lab/02_join_revenue_tasks.sql -> JOIN, revenue, summary
3. lab/03_modeling_workshop.md -> grain i walidacje słowami
4. lab/01_ddl_dml_warmup.sql -> opcjonalny DDL/DML warm-up

teoria.md jest krótka i konkretna. Do minimum wystarczy. Po lekcji poszerzaj SQL samodzielnie w internecie.

■ Praca domowa: doczytaj

Po lekcji doczytaj szczególnie:

- JOIN,
- GROUP BY,
- grain,
- walidacje,
- CTE.

■ Praca domowa: finalny homework

Oddaj:

Oddajesz finalny homework:

Te pliki stworzysz samodzielnie:

```
homework/lesson_01/  
├── 01_solution.sql  
├── 02_validation_checks.sql  
├── 03_grain_map.md  
├── 04_data_product_brief.md  
└── 05_interview_answers.md
```

■ Praca domowa: laby i PR

Oddajesz też rozwiązane laby:

```
student/lab/04_sql_basics_extra_tasks.sql  
student/lab/02_join_revenue_tasks.sql  
student/lab/03_modeling_workshop.md  
student/lab/01_ddl_dml_warmup.sql # opcjonalnie
```

Nie oddajesz `student/lab/lesson 01.db`.

Prace oddajesz przez pull request na GitHubie. Tam dostaniesz review i poprawki.

■ Pytanie na rozmowie

Pytanie na rozmowie:

```
Co może pójść nie tak przy joinie orders i order_items?
```

